

IMPLEMENTATION PROGRESS REPORT

Hostel Buddy

Smart Hostel Complaint Management System

30% · 50% · 80%

Group 19

Department of Computer Science and Engineering

IIIT Kottayam

Status: 80% implementation complete

1. Team Members

#	Member Name	Roll Number
1	Pranjal Tyagi	2024BCD0053
2	Dinesh Saini	2024BCD0033
3	Ajinkya	2024BCS0173
4	Saurabh	2024BCS0185
5	Vijay	2024BCS0249

2. Purpose of This Report

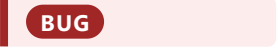
This report records how **Hostel Buddy** was built from an empty repository to a working system that is **80% complete**, and it is organised around the three implementation milestones set for the project: **30%**, **50%** and **80%**.

The project is tracked through **four components**, which are the deliverables named for the milestones. Each one is followed through all three milestones, so it is always clear what state a given part of the system was in and what was added to it next:

Component	What it covers
Git repository	Version control, branching discipline, commit structure, continuous integration
Project skeleton	The folder and layer structure — where each kind of code lives and what may call what
Database setup	The schema, its constraints, the seed data and how the schema is applied
Login module	Registration, authentication, tokens, roles and every access guard built on them

For each milestone the report gives the **phases completed**, what was done to each of the four components, **which part of each module** was finished, the **problems we faced** with their root causes and fixes, and how the milestone was verified. Section 10 then shows the application actually running, screen by screen.

2.1 How to read this document

Marker	Meaning
	Built, tested and committed at that milestone.
	Working, but extended at a later milestone.
	Not begun at that milestone.
	A defect that reached working code and had to be fixed.
	An obstacle or design dilemma — not a defect, but it cost us time.
	What we carried forward into the next milestone.

2.2 Development model

We worked in an **incremental, phase-wise model**. The project was cut into numbered phases before any code was written, each ending in a runnable system with its own tests, and nothing was merged until the

whole suite passed. This is why any milestone can be demonstrated on demand: there is no point in the history where the application does not start.

WHY NOT WATERFALL

Our requirements changed mid-project — the instructor supplied a revised ER model with three user roles after the two-role system was already working. A waterfall plan would have made that a crisis. Because every layer was isolated and covered by tests, the change was carried out as seven small phases with the suite green at the end of each one.

3. Milestone Overview

80% complete — the 30%, 50% and 80% milestones are delivered; the final 20% is listed in Section 11

3.1 Phases completed at each milestone

Milestone	Phases	What those phases produced
30%	Phase 0 — Project Foundation Phase 1 — Authentication & Accounts	A repository, the layered skeleton, the database applied at start-up, and a complete login module with role guards
50%	Phase 2 — Complaint Core Review pass Phase 3 — Admin Complaint Management Phase 4 — Dashboards & Analytics Phase 5 — Frontend UI	The complaint lifecycle, the staff queue with search and status workflow, statistics for both roles, and the complete web interface over all of it
80%	Phase 6 — Hardening & Docs Audit pass Phases A–E, G — schema v2 migration Phase F — video attachments Fix batches 1–2	Security hardening, an audit that found fifteen defects, the integration test harness and CI, the three-role multi-hostel model, and video on complaints

3.2 The four components across the three milestones

Component	At 30%	At 50%	At 80%
Git repository	Initialised; secrets excluded; one commit per phase	Phase-per-commit continued; design documents added	Continuous integration runs the suite on every push; migration done as seven separate commits
Project skeleton	Four layers created: config, database, middleware, modules	Complaints and dashboard modules added; the whole front end added	Hostels module, test harness and the page-guard layer added
Database setup	Two tables applied at start-up with constraints and foreign keys	Complaint table completed; indexes and a repeatable seed added	Migrated to six tables with an ISA hierarchy and hostel scoping
Login module	Register, login, hashing, tokens, two roles	Guards applied to every new route; profile management	Rebuilt on three roles ; managers provisioned, not self-registered; guards verified against the server

3.3 The system as it stands

6 database tables	3 user roles	17 API endpoints	12 web pages	265 automated checks	0 failing checks
-----------------------------	------------------------	----------------------------	------------------------	--------------------------------	----------------------------

3.4 Technology used

Layer	Technology	Why
Front end	HTML5, CSS3, vanilla JavaScript, Chart.js	No build step, so any team member can open the project and run it

Back end	Node.js + Express, layered routes → controllers → services → repositories	Each layer has one job, which is what made the later migration survivable
Database	Node's built-in <code>node:sqlite</code>	No native compiler needed — see Problem P-1
Authentication	JWT + bcrypt, role-based access control	Stateless, matching the scaling requirement in the SRS
Security	Helmet headers with CSP, rate-limited login, byte-level upload checks	Added in hardening and strengthened by the audit
Testing	Integration harness over real HTTP, GitHub Actions	Exercises the running API, not mocks

30% Git Repository · Project Skeleton · Login Module · Database Setup

Phase 0 — Project Foundation · Phase 1 — Authentication & Accounts

4.1 Phases completed at this milestone

Phase	Module worked on	Part of the module completed
Phase 0 Project Foundation	Application core — <code>config</code> , <code>db</code> , <code>middleware</code>	Express application assembled; typed configuration object; fixed category and status constants; database connection; schema applied at start-up; central error handler; request logger; <code>GET /api/health</code>
Phase 1 Authentication & Accounts	<code>modules/auth</code> and <code>modules/users</code>	User repository; registration and login services; password hashing; token issue and verification; <code>requireAuth</code> and <code>requireRole</code> ; seeded administrator; profile read and update; administrator's student list

4.2 Git repository

- Repository initialised and pushed, with `.gitignore` excluding secrets, the database file, uploaded files and `node_modules`.
- `.env.example` committed in place of the real `.env`, so a teammate knows exactly which variables to set without a secret ever entering history.
- One commit per phase**, each with a message describing what changed and why, so the history reads as a build log rather than a dump of work.

4.3 Project skeleton

A four-layer structure was created and has not needed to change since:

Folder	Responsibility
<code>src/config/</code>	Environment variables read once into a typed object; fixed enumerations
<code>src/db/</code>	The single database connection, the schema, and the seed script
<code>src/middleware/</code>	Cross-cutting concerns: authentication, error handling, logging, uploads, security headers
<code>src/modules/</code>	One folder per feature, each containing its routes → controller → service → repository
<code>src/utils/</code>	Token helpers and validation predicates shared across modules

THE RULE THAT HOLDS IT TOGETHER

Only a **repository** may write SQL, and only a **service** may hold a business rule. A controller does nothing but translate HTTP into a service call. This is what allowed the database model to be replaced at 80% without rewriting the application.

4.4 Database setup

- `schema.sql` is applied automatically when the server starts and is written to be **idempotent**, so a fresh clone needs no separate migration step.
- Two tables at this stage — `users` and `complaints` — matching the original design.

- Category and status columns carry SQL `CHECK` constraints, so an invalid value is refused by the database even if application code has a bug.
- Foreign keys enforced with `PRAGMA foreign_keys = ON`; write-ahead logging enabled for better read concurrency.

4.5 Login module

Part	What it does	State
Registration	Validates the input, rejects a duplicate email, hashes the password with bcrypt, creates the account	Complete
Login	Verifies the password against the stored hash and issues a signed token carrying the user id and role	Complete
Token verification	<code>requireAuth</code> rejects a missing, malformed, expired or forged token	Complete
Role guard	<code>requireRole</code> restricts a route to named roles	Partial — two roles; rebuilt for three at 80%
Seeded administrator	Created on first start from environment variables, so the system is usable immediately	Complete
Profile	Read and update own account; email and role are not editable	Complete

4.6 Module completion at this milestone

Module	Status at 30%
Authentication	Complete for the two-role model
Users / profile	Complete
Complaints	Not started — table exists, no endpoints
Dashboards	Not started
Hostels	Not started — introduced by the revised model at 80%
Front end	Not started — API only at this stage

4.7 Problems faced, and how we solved them

PROBLEM P-1 — The database library would not install.

What happened: Our technical design named `better-sqlite3`. Installing it starts a native C++ build through `node-gyp`, which failed on a team machine without Visual Studio build tools. A group project graded on several different machines cannot depend on a compiler toolchain being present.

How we solved it: Switched to Node's built-in `node:sqlite`, which offers the same synchronous prepared-statement API with no native compilation. The SQL and the schema did not change at all — only the connection module.

Trade-off we accepted: `node:sqlite` is still marked experimental and prints a warning, which we silence in the npm scripts. Because all SQL is confined to the repository layer, moving back to `better-sqlite3` or on to PostgreSQL would touch two files.

BUG B-2 — The login page told attackers which emails exist.

What happened: Signing in with an unknown address returned "no such user", while a known address with the wrong password returned "wrong password". Anyone could therefore discover which email addresses had accounts simply by trying them — a user-enumeration weakness.

How we solved it: Both failures now return the same generic `401 Invalid credentials`.

Lesson: Security messages should be deliberately vague; ordinary usability messages should be specific. Knowing which of the two you are writing is the whole skill.

PROBLEM P-3 — Where should business rules live?

The dilemma: The rule "a student may only edit a complaint while it is Pending" could be checked in the route handler in two lines. We debated this properly, because it sets the pattern for the entire codebase.

What we decided: Enforce it in the **service layer**. A route-level check protects one route; if a second entry point ever calls the same operation — a bulk action, an admin tool, a test helper — a route-only guard is silently bypassed.

Why it mattered later: This decision is the reason the three-role migration at 80% was possible at all. Because every rule already lived in the services, changing who may do what meant changing services, not hunting through routes.

PROBLEM P-4 — The seeded administrator silently failed to appear.

What happened: `.env` is deliberately not committed. On a fresh clone the admin password variable was therefore unset, the seed quietly created nothing, and the next person could not log in — with no error explaining why.

How we solved it: Added a start-up check that prints a clear warning when a required variable is missing, shipped `.env.example` as a template, and made the application **refuse to start in production** on default credentials or a default token secret.

4.8 Verification at 30%

An integration suite of **24 checks** over the running API, all passing: registration validation, duplicate-email rejection, the password hash never being returned to the client, login success and failure, token verification and expiry, and the role guard refusing a student on an administrator route.

50% Major Modules Functional

Phases 2 – 5 · Complaint Core · Admin Management · Dashboards · Frontend

5.1 Phases completed at this milestone

Phase	Module worked on	Part of the module completed
Phase 2 Complaint Core	modules/complaints (student side)	Complaint repository; create with the status forced to Pending; list own complaints; view one; edit and delete guarded as owner-only and Pending-only; image upload to disk with orphan cleanup
Review pass	Seed, configuration, error handling	Repeatable demo dataset; production refuses insecure defaults; duplicate-email race handled cleanly; upload and database decisions written down
Phase 3 Admin Complaint Management	modules/complaints (staff side)	Full queue joined with the student's name and room; search by complaint id, name or room; category and status filters; pagination; status update with remarks; resolution date recorded once
Phase 4 Dashboards & Analytics	modules/dashboard	Student statistics by state; staff statistics with totals, a zero-filled category breakdown and recent activity; all aggregation performed in SQL
Phase 5 Frontend UI	public/	Twelve pages built to the SRS wireframes; shared design system; session handling and page guards; one network layer; charts rendered from the dashboard endpoints

5.2 Git repository

- Phase-per-commit discipline continued, with a separate commit for the review pass so the fixes are distinguishable from the feature work.
- The formal design documents were added to the repository alongside the code, so the design and the implementation are versioned together.

5.3 Project skeleton

The skeleton was extended, never restructured — every addition followed the pattern set at 30%:

Added	Contents
src/modules/complaints/	Routes, controller, service and repository for the whole complaint lifecycle
src/modules/dashboard/	A service that composes the complaint and user repositories; no SQL of its own
src/middleware/upload.js	File upload handling with unique names and a size limit
public/	The front end: pages, one stylesheet, and one script per page over a shared session, network and interface layer

5.4 Database setup

- The complaint table was completed with the image path, status, administrator remarks, and created, updated and resolved timestamps.
- Indexes added on the columns the queue actually filters and sorts by, so search and pagination stay fast as the table grows.
- A **repeatable seed script** was written so any team member can demonstrate the product in a known state; it is safe to re-run and leaves existing rows alone.

5.5 Login module

No new authentication was written at this milestone — instead the guards built at 30% were **applied to everything new**, which is what proved they were built in the right place:

- Every student route sits behind `requireAuth` plus the student role.
- Every staff route sits behind `requireAuth` plus the administrator role.
- Ownership — "this complaint is mine" — is checked in the service, not the route, so it holds for every caller.

5.6 Module completion at this milestone

Module	Status at 50%
Authentication	Complete for the two-role model
Users / profile	Complete
Complaints — student	Complete — create, list, view, edit, delete, image
Complaints — staff	Complete — queue, search, filters, pagination, status workflow
Dashboards	Complete for both roles
Front end	Complete — every endpoint reachable from a page
Hostels	Not started — introduced by the revised model at 80%

5.7 Problems faced, and how we solved them

BUG B-1 — Photo uploads broke every complaint submission.

What happened: The create-complaint form sends multipart data because it carries an image, but our JSON body-parser ran first and saw nothing. The request body came back empty, so validation rejected every single submission — including perfectly valid ones.

Root cause: Middleware order. The upload middleware must run *before* the fields are readable, and we had ordered the validator ahead of it.

How we solved it: Fixed the order to `upload → validate → controller`, and wrote the required order into the architecture document so nobody re-orders it by accident.

Lesson: Middleware order is part of the contract, not an implementation detail.

BUG B-3 — The resolution date kept moving.

What happened: We set the resolution date whenever the status was Resolved. So every later edit — even correcting a typo in the remarks — pushed it forward, and moving a complaint from Resolved to Closed reset it again. The date became meaningless.

How we solved it: The date is set only when it is currently empty and the complaint has reached Resolved.

Lesson: "The first time X happens" is a different rule from "whenever X is true", and the two look identical until you read them carefully.

BUG B-4 — The client could award itself a status.

What happened: The create endpoint accepted a status field from the request body. A student could therefore submit a complaint already marked *Resolved* and skip the workflow.

How we solved it: The service now **ignores** any status the client sends on create and forces Pending. Status changes only through the staff-only endpoint.

Lesson: Never accept a field from the client that the workflow owns.

BUG B-5 — Front-end pages each handled the session differently.

What happened: Pages made their own network calls. Some forgot the authorization header, some forgot the content type, and an expired session produced a blank white page instead of sending the user to log in.

How we solved it: Wrote a single network layer that attaches the token, sets headers, parses the response and redirects to login when the session is rejected. Every page goes through it and no page does its own fetching.

Lesson: Centralise the network boundary. Cross-cutting logic repeated per page will be wrong on at least one page.

PROBLEM P-5 — Should uploaded photos be access-controlled?

The dilemma: Complaint photos are served as static files. That path is not behind the ownership check that protects the complaint record, so anyone holding the URL can open the image.

What we decided: Accept it for this version and **write it down** rather than leave it undiscovered. File names are a timestamp plus random characters, so they are effectively unguessable, and the URLs only ever appear inside access-controlled pages. Gating them properly means streaming every image through an authenticated route or issuing signed storage URLs.

Lesson: An accepted weakness that is documented is a decision. The same weakness undocumented is an oversight — and a reviewer cannot tell them apart unless you write it down.

PROBLEM P-6 — A strict security policy broke the landing page.

What happened: Adding a Content-Security-Policy that allows scripts only from our own origin blocked the one inline script we had — the "skip the landing page if already signed in" check. The choice looked like: weaken the policy, or lose the feature.

How we solved it: Neither. We moved the inline script into an external file, so the strict policy stayed exactly as it was.

Lesson: Keep scripts in external files from the start. It costs nothing while you are writing them, and it is what lets you ship a strict policy later.

5.8 Verification at 50%

Four integration suites — authentication, complaints, administration and dashboards — totalling **103 checks**, all passing, plus a manual walkthrough of both user journeys in the browser with no console errors.

LESSON Reaching 50% taught us that a feature is not finished when the endpoint works. Three of the four bugs above were only visible from the front end, and one of them made the flagship feature completely unusable while every server-side test was green.

80% Integration Testing · Audit · Three-Role Migration

Phase 6 · Audit pass · Phases A–G · Phase F · Fix batches

6.1 Phases completed at this milestone

Phase	Module worked on	Part of the module completed
Phase 6 Hardening & Docs	middleware/security , documentation	Security headers with a content policy; rate limiting on the login route; documentation reconciled with the code; screenshots generated from the running application
Audit pass	Every module	Validation rewritten around type predicates; the status lifecycle enforced; uploads verified by file signature; error responses cleaned; search escaping; password length; rate limiting actually enabled; the administrator's student page built
Test harness	tests/	A runner that starts its own server against a throwaway database, six suites made self-contained and order-independent, and a regression suite pinning every audit defect
Phase A Schema design	Design only — no application code	The revised six-table model drawn, written up and verified against SQLite before anything depended on it
Phase B Schema applied	db/schema.sql , db/seed.js	Six tables created; seed rebuilt to produce hostels, managers and students
Phase C Authentication rebuilt	modules/auth , middleware/auth	Registration writes the user and its subtype in one transaction; three roles; requireStaff and requireSuperAdmin guards
Phase D Hostels	modules/hostels (new)	Public hostel list for the registration form; super-admin hostel management; manager provisioning bound to a hostel
Phase E Complaints migrated	modules/complaints , modules/dashboard	Complaints moved onto the new model and scoped by hostel in SQL; every dashboard figure narrowed the same way
Phase G Frontend migrated	public/	Registration takes a roll number and hostel; hostel column and management page; role-aware navigation for three roles
Phase F Video attachments	middleware/upload , complaints, front end	A second attachment field with its own size limit and format allowlist, verified by file signature; a player in both detail views
Fix batches	Logging, front-end guards	Request logging reduced to what matters; the page guard moved ahead of rendering

6.2 Git repository

- **Continuous integration** added: the full suite runs on GitHub Actions on every push, so a broken commit is visible without anyone remembering to test.
- The migration was deliberately committed as **seven separate phase commits** rather than one large change, so each step is reviewable and revertible on its own.

6.3 Project skeleton

Added	Contents
src/modules/hostels/	The new module, following the same four-file pattern as every other

tests/	The runner plus six suites — authentication, hostels, complaints, administration, dashboards, regression
public/js/guard.js	The page guard, loaded before any markup so a protected page cannot render for the wrong person
.github/workflows/	The continuous-integration definition

6.4 Database setup — migrated to six tables

Part-way through this milestone the instructor supplied a revised entity-relationship model. It replaced the two-table, two-role design with **six tables and three roles**, adding hostels as first-class entities and specialising `USER` into `STUDENT`, `MANAGER` and `SUPER_ADMIN`.

Table	Holds
hostel	Name, location and capacity — the entity a manager is scoped by
user	Name, email, password hash and the role discriminator
student	Roll number, hostel and room, keyed by the same id as the user
manager	The one hostel this manager is responsible for
super_admin	Deliberately carries no hostel — being unscoped is the role
complaint	Category, description, image, video, status, remarks, timestamps, and its own hostel

6.5 Login module — rebuilt on three roles

Part	What changed	State
Registration	Now requires a roll number and a hostel; writes the user row and its student row in one transaction	Complete
Roles	Student, manager and super admin, with the role acting as the discriminator for the subtype tables	Complete
Staff guard	<code>requireStaff</code> admits a manager or a super admin to the shared screens	Complete
Super-admin guard	<code>requireSuperAdmin</code> restricts hostel and manager management	Complete
Manager provisioning	Managers are created by the super admin and bound to a hostel; there is no public sign-up	Complete
Hostel scoping	A manager's hostel is read from their own row on every request, never taken from the request or the token	Complete
Page guards	Enforced before the page renders and confirmed against the server	Complete
Session revocation	Signing out clears the browser session; invalidating the token itself is scheduled	Remaining

6.6 Module completion at this milestone

Module	Status at 80%
Authentication	Complete — three roles, ISA model
Users / profile	Complete — plus manager provisioning
Hostels	Complete — public list and super-admin management
Complaints	Complete — migrated, hostel-scoped, photo and video

Dashboards	Complete — scoped for managers, unscoped for the super admin
Front end	Complete — three-role navigation and guards
Testing	Complete — 265 checks in six suites, run on every push

6.7 Integration testing — how it works

A single command starts the server on its own port against a **throwaway database and upload directory**, waits for the health endpoint to answer, seeds the hostels and accounts the scoping tests need, runs all six suites in order, then shuts the server down and deletes both. Development data is never touched.

Suite	Checks	What it proves
authentication	47	Registration across three roles, login, token guards, profile rules
hostels	46	Public hostel list, super-admin-only management, manager provisioning
complaints	56	Student lifecycle, ownership, Pending-only edits, photo and video attachments
administration	52	Staff queue, search, filters, pagination, status transitions, hostel scoping
dashboards	23	Student and staff statistics, scoped aggregation
regression	41	Every audit defect, pinned so it cannot come back
Total	265 checks	All passing, in any order

6.8 The audit — how we tested

After hardening we audited the whole application **against a running server**: the full suite, then manual probing of the API with malformed and hostile input, then a browser walkthrough of every page at desktop and mobile widths. **237 probe requests produced zero server errors**, and SQL injection, cross-site scripting, token forgery, mass assignment, ownership isolation and path traversal all held. Fifteen things did break; the five most serious follow, and the full list is in Section 8.

BUG B-8 — A JSON object was stored as the text "[object Object]".

What happened: Registering with an object in place of the name returned success and created an account literally named [object Object]. The same worked for a complaint description and for staff remarks, so the junk was visible on the staff complaint table.

Root cause: Every validator converted first and checked second. Converting an object to text produces a non-empty string, so it sailed through. The helper that would have caught this existed in our validators file and **was exported but never called anywhere**.

How we solved it: Rewrote validation around explicit type predicates, used at every entry point. A value of the wrong type is now rejected, never converted.

Lesson: Dead code in a validation module is a warning sign, not clutter. Check the type *before* you touch the value.

BUG B-9 — Complaints could be Pending while carrying a resolution date.

What happened: The status endpoint checked only that the new status was one of the four valid names — not that the move was legal. Closed could go back to Pending. Since a resolution date is deliberately never cleared, rolling a Resolved complaint backwards produced a row that was *Pending with a resolution date*, and the student could then edit a complaint that had already been resolved.

Root cause: The lifecycle was drawn in our diagrams and documented, but only the *vocabulary* was enforced in code, never the *ordering*.

How we solved it: A transition guard: a complaint's position may stay the same or move forward, never backward. The staff dropdown now offers only the statuses the server will accept, so the rule is visible rather than a surprise error.

Lesson: A state machine drawn in a diagram is not a state machine in the code.

BUG B-12 — Upload checking trusted the browser's word.

What happened: We checked the file type using the type declared in the upload header — which is simply whatever the client writes there. A text file declared as an image was accepted, stored with an image extension, and served back as an image.

How we solved it: After the file is saved we read its **first bytes and identify the real format from its signature**. Anything that is not genuinely one of the accepted formats, or whose bytes disagree with its declared type, is deleted and rejected.

Lesson: Anything the client sends is a claim, not a fact. If a decision depends on what a file *is*, read the file.

BUG B-10 — Our own tests were polluting the demo database.

What happened: The test command assumed a server was already running and crashed when it was not. Worse, the suites hit the development server, wrote into the real database and never cleaned up — after a few runs the staff screen was full of test accounts. One test even opened the database file directly, reaching around the API entirely.

How we solved it: Built the harness described in §6.7. The test that poked the database now advances the complaint through the staff endpoint like a real user.

Lesson: A test that needs a human to set something up first will eventually be run without it. And an integration test that reaches around the API is testing the database, not the application.

BUG B-16 — Brute-force protection was switched off in the only environment we run.

What happened: The login rate limiter was replaced by a do-nothing function unless the environment was production — and the project is demonstrated in development. The protection we had designed, documented and presented in our architecture review **had never actually run once**.

How we solved it: The limiter is now active in development and production alike and disabled only under the test runner. It counts *failed* attempts, so a real user signing in repeatedly is never locked out while a password guesser still is.

Lesson: Security that only exists in an environment you never run is not security.

THE MAIN LESSON OF THIS MILESTONE Every one of the fifteen audit defects was found by **running** the application against hostile input — not by reading the code, and not by the test suite. The suites were green the entire time. They tested the paths we designed, which is exactly the input an audit must not limit itself to.

PROBLEM P-7 — A change of data model with a working system on top of it.

The risk: The revised model touched the schema, the seed, authentication, every complaint query, every dashboard figure, all guards and most of the front end. Done as one large change it would have left us with a broken application and no way to tell which part broke it.

How we handled it: Split into **seven phases**, each a separate commit with the full suite green before the next began. Phase A produced only a design document and a verified schema — no application code at all — so the model could be checked against the instructor's diagram before anything depended on it.

Result: The migration completed without the application being broken at any commit.

BUG B-18 — Signed-out users could still see the dashboard.

What happened: Signing out and reloading still showed the staff dashboard. The guard was real, but it ran from the bottom of the page — after all the markup had been parsed and drawn, and after a large chart library had downloaded. The redirect always came; it just came last.

How we solved it: The check now runs from the document head, before the stylesheet and before any markup, so a failed check navigates away without the protected page ever being drawn. It is also re-run when the browser restores a page from its back-forward cache, and confirmed against the server so a role edited in the browser cannot be trusted.

Lesson: A guard that runs after the page is drawn is a redirect, not a guard.

7. The Implemented Data Model

Four decisions in this model are worth calling out, because each one moves a rule out of the application and into the database, where it holds regardless of whether our code is correct:

- **Shared primary key.** Each subtype's identifier is both its primary key and its foreign key to `USER`, so one person cannot hold two identities.
- **A complaint references `STUDENT`, not `USER`.** The database itself refuses a complaint raised by a manager.
- **A complaint stores its own hostel.** It records the hostel the complaint was raised *against*, at that time, so history stays truthful if a student later changes hostel — and scoping becomes one indexed column instead of a join.
- **The super admin deliberately has no hostel.** Being unscoped is the point of the role, and the schema says so.

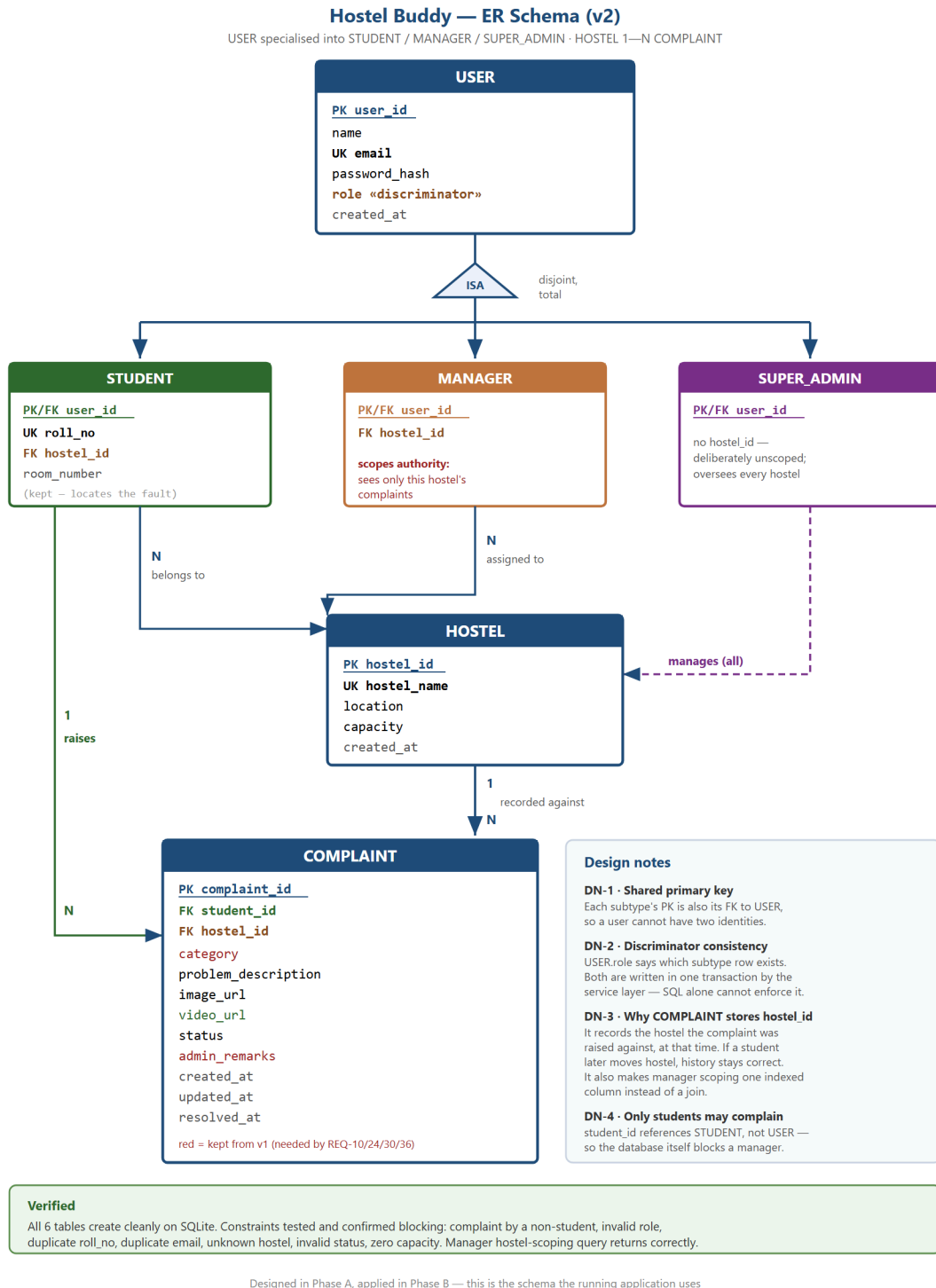


Figure 1 — The implemented entity-relationship model: *USER* specialised into *STUDENT*, *MANAGER* and *SUPER_ADMIN*, with *HOSTEL* 1—N *COMPLAINT*.

8. Every Defect at a Glance

All twenty-three problems and defects encountered so far, with the milestone at which each was found and how it was resolved. Entries marked **R** are pinned by a check in the regression suite, so they cannot return unnoticed.

Ref	Problem	Root cause and fix	Found at	R
P-1	Database library needed a C++ compiler	Native build failed without Visual Studio; switched to Node's built-in SQLite	30%	—
B-2	Login revealed which emails exist	Two different error messages; now one generic rejection	30%	—
P-3	Where to enforce business rules	Decided on the service layer, not routes — what made the later migration possible	30%	—
P-4	Seeded administrator silently missing	Unset environment variable; added start-up warning and production refusal	30%	—
B-1	Every complaint submission failed with a photo	Middleware ordered validator before upload; order corrected and documented	50%	—
B-3	Resolution date kept moving	Unconditional assignment; now set once, on first reaching Resolved	50%	R
B-4	Client could set its own status	Accepted a client field the workflow owns; now forced to Pending	50%	R
B-5	Inconsistent sessions and blank pages on expiry	Ad-hoc calls per page; centralised into one network layer	50%	—
P-5	Uploaded photos not access-controlled	Accepted for this version with unguessable names; documented, not hidden	50%	—
P-6	Security policy blocked the landing page	Moved the inline script to a file instead of weakening the policy	50%	—
B-6	Tests passed alone, failed together	Shared state and absolute counts; every suite now owns its data and asserts deltas	80%	R
B-8	Object stored as "[object Object]"	Validators converted before checking; rewritten around type predicates	80%	R
B-9	Status could move backwards	Only vocabulary enforced, not ordering; added a transition guard	80%	R
B-10	Tests wrote into the demo database	No isolation; built a harness with a throwaway database and server	80%	R
B-11	A requirement was built but unreachable	Endpoint existed with no page; added the student list screen and nav link	80%	R
B-12	Upload type trusted the browser	Declared type treated as fact; now verified by file signature	80%	R
B-13	Framework errors leaked to users	Handler returned raw messages; only our own errors keep their text	80%	R
B-14	"%" in search matched everything	Parameterisation protects the parser, not the pattern language; escaped it	80%	R
B-15	Long passwords silently truncated	The hashing library reads 72 bytes; longer passwords now rejected clearly	80%	R
B-16	Rate limiting never actually ran	Enabled only in production, which we never run; now on everywhere but tests	80%	R
B-17	Six small correctness and layout defects	Page clamping, image removal, styled 404, dialog timer, mobile actions, logging	80%	R
P-7	Data model changed under a working system	Split into seven phases, suite green at each; design verified before any code	80%	—

B-18	Signed-out users still saw the dashboard	Guard ran after the page was drawn; moved into the head, before any markup	80%	—
------	--	--	-----	---

8.1 Where our defects came from

How it was found	Entries	What that tells us
Auditing the running application with hostile input	11	By far the most productive technique. Counting B-17's six grouped items individually, the audit accounted for 15 separate defects — and the test suite was green before it started and green after it finished
Building the front end over a finished API	3	A working endpoint is not a working feature; one of these made the flagship feature unusable while every server test passed
Ordinary development and code review	3	Caught early and cheaply, before anyone else depended on them
Design debates held before writing the code	6	Cheapest of all — decisions and trade-offs that never became defects
Total 23 entries		All resolved

9. Test Cases Prepared

A representative selection from the 265 automated checks. Each names its precondition, the action and the result the server must produce; all of them run over real HTTP against a real database, not against mocks.

9.1 Authentication and accounts

ID	Test case	Input	Expected result
TC-A1	Register a valid student	Name, email, password, roll number, hostel	Created, with a token; no password hash in the response
TC-A2	Reject a duplicate email	An email already registered	Conflict, <code>EMAIL_TAKEN</code>
TC-A3	Reject a duplicate roll number	A roll number already registered	Conflict, <code>ROLL_NO_TAKEN</code>
TC-A4	Reject a non-string name	An object in place of the name	Rejected — never stored as text
TC-A5	Do not reveal unknown emails	Unknown email vs wrong password	Identical rejection for both
TC-A6	Reject a forged token	A token signed with the wrong secret	Unauthenticated
TC-A7	Self-registration cannot create staff	A staff role supplied in the body	Account created as a student regardless
TC-A8	Reject an over-long password	A password above the hashing limit	Rejected with an explanatory message

9.2 Complaints and the lifecycle

ID	Test case	Input	Expected result
TC-C1	Create a complaint	Valid category and description	Created, status forced to Pending
TC-C2	Attach a photo and a video	A real image and a real video file	Created; both stored and served
TC-C3	Reject a disguised file	A text file declared as an image	Rejected; the file is deleted
TC-C4	Reject an unplayable video	A format the browser cannot play	Rejected — not on the allowlist
TC-C5	Enforce the per-field size limit	An image above the image limit	Rejected, <code>FILE_TOO_LARGE</code>
TC-C6	Owner-only access	Student B opens Student A's complaint	Forbidden
TC-C7	Edit only while Pending	Edit a complaint already In Progress	Refused, <code>NOT_PENDING</code>
TC-C8	Status cannot move backwards	Closed → In Progress	Refused, <code>INVALID_TRANSITION</code>
TC-C9	Resolution date is set once	Resolved, then remarks edited	Date unchanged by the second edit
TC-C10	No open complaint has a resolution date	Query the whole table	Zero such rows
TC-C11	Deleting removes the attachments	Delete a complaint with both files	Row gone; both files no longer served

9.3 Roles and hostel scoping

ID	Test case	Input	Expected result
----	-----------	-------	-----------------

TC-S1	Manager sees only their hostel	Manager lists all complaints	Only their own hostel's rows
TC-S2	Manager cannot read another hostel	Open a complaint from another hostel	Forbidden
TC-S3	Manager cannot modify another hostel	Change its status	Forbidden; complaint verifiably untouched
TC-S4	Scope cannot be widened by searching	Search another hostel's complaint id	No results
TC-S5	Manager cannot create hostels	Manager posts a new hostel	Forbidden — would let them redraw their own authority
TC-S6	Manager cannot provision managers	Manager creates a manager account	Forbidden
TC-S7	A manager must have a hostel	Provision one with no hostel	Rejected as invalid
TC-S8	Students cannot reach staff routes	Student calls the queue and staff dashboard	Forbidden on both
TC-S9	A hostel in use cannot be deleted	Delete a hostel that still has students	Refused, <code>HOSTEL_IN_USE</code>

9.4 Robustness and security

ID	Test case	Input	Expected result
TC-R1	Search wildcards are literal	Search for the wildcard characters	No match-everything behaviour
TC-R2	Page numbers are clamped	A page number past the end	Clamped to the last real page
TC-R3	No framework internals leak	Malformed request body	Our own message and code
TC-R4	Unknown pages get our own 404	A mistyped address	The styled page; JSON for API routes
TC-R5	Student list is staff-only	Student requests it	Forbidden; no password hashes ever returned

UNIT-LEVEL CHECKS Alongside these, the pure functions that carry the rules — the status-transition comparison, the search escaper, the type predicates and the upload signature detector — are each exercised directly by the cases above, since every one of them was written in response to a specific defect.

10. The Product Running Now

The figures in this section were captured from the application **running on a live server with the demo data loaded**, at the state of the code described by this report. They are not mockups or wireframes. Before capture the full suite was run: **265 checks, all passing**.

The demo data is three hostels, one super admin, three managers, six students and thirteen complaints spread across all four states.

10.1 Entry and registration

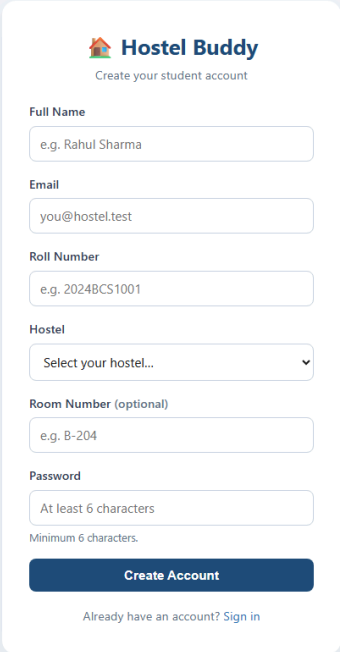
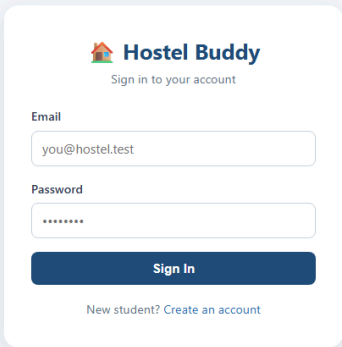
The image shows a registration form for 'Hostel Buddy'. The form is titled 'Create your student account' and includes fields for 'Full Name' (with example 'e.g. Rahul Sharma'), 'Email' (with example 'you@hostel.test'), 'Roll Number' (with example 'e.g. 2024BCS1001'), 'Hostel' (a dropdown menu with 'Select your hostel...' and a downward arrow), 'Room Number (optional)' (with example 'e.g. B-204'), and 'Password' (with a hint 'At least 6 characters' and a note 'Minimum 6 characters'). A dark blue 'Create Account' button is at the bottom, followed by a link 'Already have an account? Sign in'.

Figure 2 • Registration. A student signs up with a name, email, password, **roll number** and the **hostel they live in**, chosen from a live list loaded from the server. Registration always creates a student — a staff role supplied in the request is ignored, because manager accounts are provisioned by the super admin and never self-registered.



The login form for Hostel Buddy is centered on a light blue background. It features the Hostel Buddy logo at the top, followed by the text "Sign in to your account". Below this are two input fields: "Email" with the placeholder text "you@hostel.test" and "Password" with masked characters "*****". A dark blue "Sign In" button is positioned below the password field. At the bottom of the form, there is a link that says "New student? Create an account".

Figure 3 · Login. One form for all three roles; the server decides which and sends the user to the right home screen. An unknown email and a wrong password produce the same message, so the page cannot be used to discover which accounts exist. Repeated failures from one address are rate-limited.

10.2 What a student can do

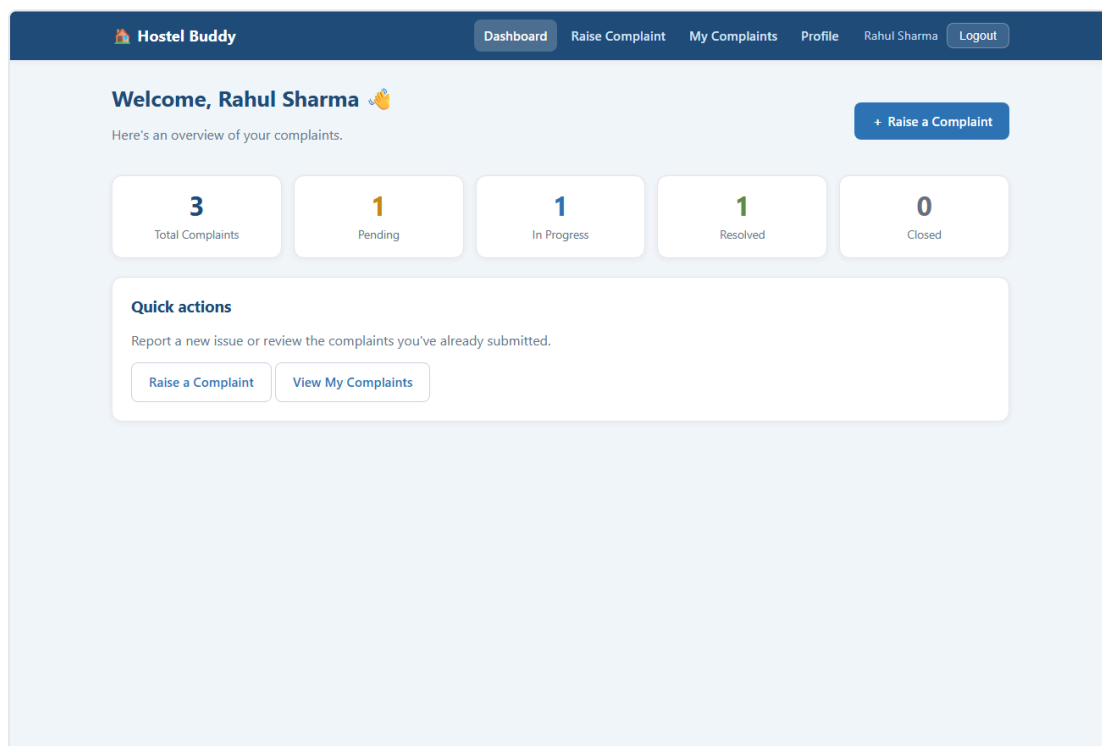


Figure 4 · Student dashboard. The student's own complaints counted by state — total, pending, in progress, resolved and closed. These figures come from a database aggregation scoped to this student; no other student's complaints are ever counted here.

Hostel Buddy

DashboardRaise ComplaintMy ComplaintsProfileRahul SharmaLogout

Raise a Complaint

Category

Select a category...

Description

Describe the issue in detail...

0/1000 characters

Attach an image (optional)

Choose FileNo file chosen

PNG, JPEG or WEBP, up to 5 MB.

Attach a video (optional)

Choose FileNo file chosen

MP4 or WEBM, up to 30 MB. Useful for faults you can only show in motion — a flickering light or a dripping pipe.

Submit Complaint

Cancel

Figure 5 • Raising a complaint. A category from the fixed list, a description with a live character counter, and two optional attachments — a **photo** and a **video**, each with its own size limit. Both are checked by their actual file signature on the server, so a file that merely claims to be an image or a video is refused. The complaint's hostel is taken from the student's own record, never from the form, so nobody can file against a hostel they do not live in.

Hostel Buddy

DashboardRaise ComplaintMy ComplaintsProfileRahul SharmaLogout

My Complaints

+ Raise a Complaint

ID	CATEGORY	DESCRIPTION	STATUS	SUBMITTED	ACTIONS
#1	Wi-Fi	No Wi-Fi signal in room B-204 since morning.	Pending	Aug 31, 2026	ViewEditDelete
#2	Electricity	Ceiling fan not working.	In Progress	Aug 28, 2026	View
#3	Furniture	Study chair is broken.	Resolved	Aug 20, 2026	View

Figure 6 • My Complaints. Every complaint this student has raised, with its current status and the staff remarks. **Edit** and **Delete** appear only while a complaint is still Pending — once a manager has started work the buttons are gone, and the server refuses the operation even if the request is made directly.

10.3 Hostel scoping — the same screen, two roles

This pair is the clearest demonstration of the three-role model. Both figures are the **same page, from the same code**. The only difference is who is signed in.

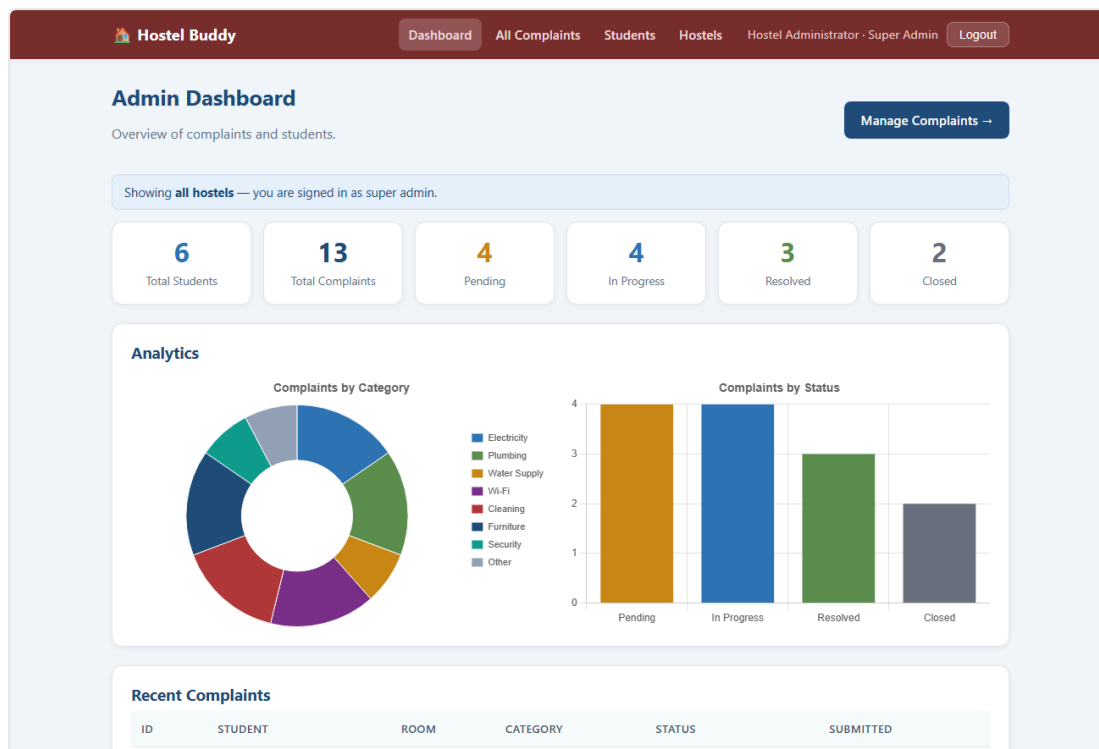


Figure 7 • Dashboard as the super admin. The banner reads "Showing **all hostels**". Six students, thirteen complaints — the whole institution, with the category and status charts covering every hostel.

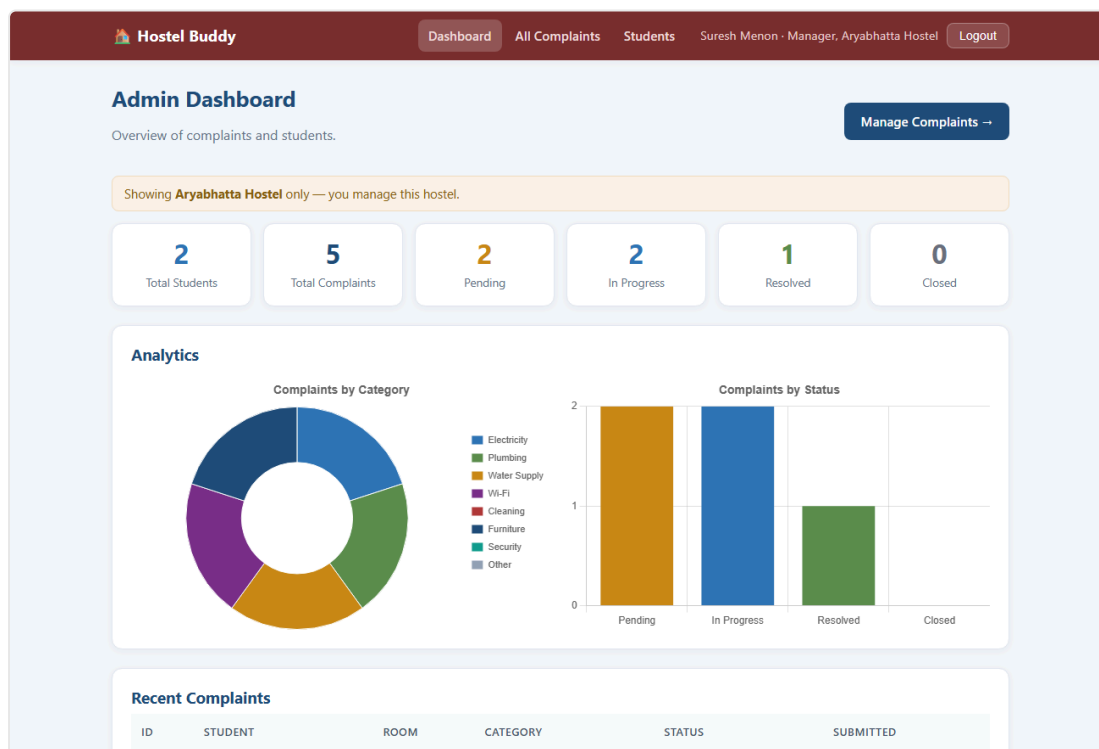


Figure 8 • The same dashboard as a hostel manager. The banner now reads "Showing **Aryabhata Hostel** only — you manage this hostel". Two students, five complaints; every chart redrawn from that hostel alone. The navigation has also lost the **Hostels** link, which only the super admin may use. The narrowing happens in SQL, so the other hostels' rows are never loaded — they are not fetched and then hidden.

10.4 Working the complaint queue

Hostel Buddy								
Dashboard All Complaints Students Hostels Hostel Administrator · Super Admin Logout								
All Complaints								
Search Category Status								
Complaint ID, student name, or room number... All All Search Clear								
ID	STUDENT	HOSTEL	ROOM	CATEGORY	DESCRIPTION	STATUS	SUBMITTED	ACTION
#8	Priya Singh	Ramanujan Hostel	B-210	Wi-Fi	Wi-Fi very slow in the evening.	Pending	Aug 31, 2026	Manage
#1	Rahul Sharma	Aryabhata Hostel	B-204	Wi-Fi	No Wi-Fi signal in room B-204 since morning.	Pending	Aug 31, 2026	Manage
#10	Amit Kumar	Bhaskara Hostel	D-305	Cleaning	Washroom needs deep cleaning.	Pending	Aug 30, 2026	Manage
#4	Ananya Verma	Aryabhata Hostel	A-112	Plumbing	Leaking tap in the washroom on A-wing.	Pending	Aug 30, 2026	Manage
#6	Karan Nair	Ramanujan Hostel	C-007	Security	Main gate CCTV appears to be off.	In Progress	Aug 29, 2026	Manage
#2	Rahul Sharma	Aryabhata Hostel	B-204	Electricity	Ceiling fan not working.	In Progress	Aug 28, 2026	Manage
#11	Amit Kumar	Bhaskara Hostel	D-305	Plumbing	Flush not working in D-305.	In Progress	Aug 27, 2026	Manage
#5	Ananya Verma	Aryabhata Hostel	A-112	Water Supply	No hot water in the mornings.	In Progress	Aug 26, 2026	Manage
#12	Sneha Rao	Bhaskara Hostel	D-110	Furniture	Cupboard door hinge is loose.	Resolved	Aug 22, 2026	Manage

Figure 9 · The complaint queue as the super admin. Every complaint in every hostel, with a **Hostel** column, free-text search over complaint id, student name, roll number and room, category and status filters, and pagination. **Manage** opens the complaint, shows its photo and video, and offers only the statuses the server will accept — the lifecycle moves forward only.

Hostel Buddy

Dashboard

All Complaints

Students

Suresh Menon · Manager, Aryabhata Hostel

Logout

All Complaints

Search

Complaint ID, student name, or room number...

Category

All

Status

All

Search

Clear

ID	STUDENT	ROOM	CATEGORY	DESCRIPTION	STATUS	SUBMITTED	ACTION
#1	Rahul Sharma	B-204	Wi-Fi	No Wi-Fi signal in room B-204 since morning.	Pending	Aug 31, 2026	Manage
#4	Ananya Verma	A-112	Plumbing	Leaking tap in the washroom on A-wing.	Pending	Aug 30, 2026	Manage
#2	Rahul Sharma	B-204	Electricity	Ceiling fan not working.	In Progress	Aug 28, 2026	Manage
#5	Ananya Verma	A-112	Water Supply	No hot water in the mornings.	In Progress	Aug 26, 2026	Manage
#3	Rahul Sharma	B-204	Furniture	Study chair is broken.	Resolved	Aug 20, 2026	Manage

« Prev

Page 1 of 1 · 5 complaints

Next »

Figure 10 · The same queue as a hostel manager. Five rows instead of thirteen, all Aryabhata. A manager who requests another hostel's complaint directly — by id, by search, or by attempting a status change — is refused. That refusal is tested in both directions, and the complaint is verified afterwards to be untouched.

10.5 Administration

Hostel Buddy					
Dashboard All Complaints Students Hostels Hostel Administrator · Super Admin Logout					
Students					
Every student account registered on Hostel Buddy.					
Search Name, email, or room number... Clear					
ROLL NO	NAME	EMAIL	HOSTEL	ROOM	REGISTERED
2024BCS1006	Sneha Rao	sneha@hostel.test	Bhaskara Hostel	D-110	Sep 1, 2026
2024BCS1005	Amit Kumar	amit@hostel.test	Bhaskara Hostel	D-305	Sep 1, 2026
2024BCS1004	Priya Singh	priya@hostel.test	Ramanujan Hostel	B-210	Sep 1, 2026
2024BCS1003	Karan Nair	karan@hostel.test	Ramanujan Hostel	C-007	Sep 1, 2026
2024BCS1002	Ananya Verma	ananya@hostel.test	Aryabhatta Hostel	A-112	Sep 1, 2026
2024BCS1001	Rahul Sharma	rahul@hostel.test	Aryabhatta Hostel	B-204	Sep 1, 2026
Showing 6 of 6 students					

Figure 11 · Registered students. Name, email, roll number, hostel and room, filterable from the browser. Password hashes are never included in the response. A manager sees only the students of their own hostel; the super admin sees all of them.

Hostel Buddy					
Dashboard All Complaints Students Hostels Hostel Administrator · Super Admin Logout					
Hostels & Managers					
Add hostels and assign a manager to each one.					
+ Add Hostel					
Hostels					
NAME	LOCATION	CAPACITY	MANAGERS	ACTIONS	
Aryabhatta Hostel	North Campus	200	1	Edit	Delete
Bhaskara Hostel	East Campus	150	1	Edit	Delete
Ramanujan Hostel	South Campus	180	1	Edit	Delete
+ Add Manager					
Managers					
NAME	EMAIL	MANAGES	ADDED		
Suresh Menon	manager.aryabhatta@hostel.test	Aryabhatta Hostel	Sep 1, 2026		
Vikram Bose	manager.bhaskara@hostel.test	Bhaskara Hostel	Sep 1, 2026		
Deepa Iyer	manager.ramanujan@hostel.test	Ramanujan Hostel	Sep 1, 2026		

Figure 12 · Hostels and managers — super admin only. Hostels are created, renamed and removed here, and manager accounts are created and bound to one hostel each. A hostel that still has students, managers or complaints cannot be deleted; the server explains what is blocking it rather than silently orphaning rows. A manager who opens this page is sent back to their dashboard, and every button on it is refused by the server as well — a manager must not be able to edit the structure that scopes their own authority.

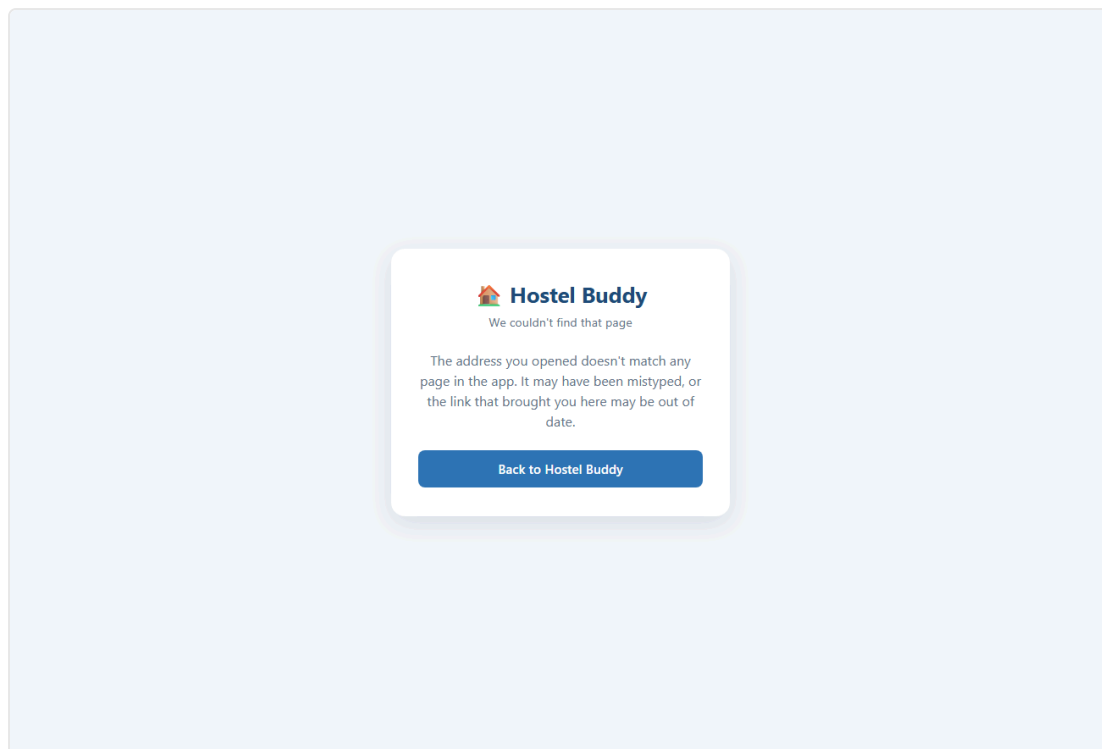


Figure 13 • A mistyped address. The application's own page rather than the framework's default error, so a wrong link never drops the user out of the product. API routes return the same condition as JSON.

10.6 What is demonstrably working

Capability	Evidence in this section
Registration with roll number and hostel	Figure 2 — the hostel list is loaded live from the server
One login for three roles	Figure 3, and each later figure is a different signed-in role
Student complaint lifecycle	Figures 4–6 — raise with attachments, track, edit while Pending
Photo and video attachments	Figure 5 — both fields, each with its own limit
Hostel scoping	Figures 7 vs 8 and 9 vs 10 — the same page, different data
Search, filters, pagination	Figures 9 and 10
Status workflow with remarks	Figure 9 — only forward transitions are offered
Analytics for both staff roles	Figures 7 and 8 — charts redrawn per scope
Hostel and manager management	Figure 12 — super admin only, in the page and on the server
Errors handled inside the product	Figure 13

11. The Remaining 20%

Item	What is left to do	Status
Session revocation	Signing out clears the browser's session, but the token itself stays valid until it expires. A server-side invalidation step is designed and scheduled.	Remaining
Super-admin scope review	Deciding whether the super admin should act on complaints directly or oversee them read-only, leaving the queue to hostel managers.	Remaining
Document revision	The SRS, the SADD and the architecture presentation still describe the earlier two-role design and must be brought onto the three-role model — ER diagram, data dictionary, class and sequence diagrams, and the traceability matrix.	Remaining
Regression coverage	Pinning the two most recent fixes with their own checks, and folding the browser-level page-guard verification into the standard suite.	Remaining
Demonstration video	A recorded walkthrough of all three roles, end to end.	Remaining
Deployment	Hosting the application so it can be reached without a local checkout.	Remaining

12. Conclusion

At 80%, Hostel Buddy is a working three-role, multi-hostel complaint management system. Every functional requirement in the SRS is implemented and reachable from the interface, the entity model matches the one supplied by the instructor, and **265 automated checks run on every push and all of them pass**. Section 10 shows the product actually running, screen by screen.

The part of this milestone we would defend most strongly is the audit. Our test suite was green before it began and green after it ended, yet the audit found fifteen defects — including one that let a complaint sit in a contradictory state, one that stored arbitrary objects as text, and one that meant the brute-force protection we had presented in our architecture review had never run at all. That gap between "the tests pass" and "the system is correct" is the most valuable thing the project has taught us, and it changed how we work: **we now try to break each feature before we call it finished**.

The remaining 20% is largely consolidation — finishing the session-revocation work, bringing the formal documents onto the current data model, and recording the demonstration. The engineering is in place.

Group 19 — Team Signature

Faculty Signature